

SC1001: Introduction to Computational Thinking

Week 6: Basics of Functions

Jordan Boyd-Graber

Nanyang Technological University

Slides

Slides and updates will be posted at

`boydgraber.org/teaching/SC_1001/`

Overview

1. Write repetitive code to apply discounts to several items
2. Identify what is repetitive in that code
3. Refactor the logic into a function
4. Compare the function version with the repeated-code version
5. Extend the program to track totals
6. Use a loop to remove hardcoded items

Q1: Discounted Prices

Write Python code to calculate discounted prices for different items.

Each item has an original price and discount rate. Compute

- discount amount
- final price

for several hardcoded items.

Q1: One Direct Solution

```
# Item 1
original_price = 120.00
discount_rate = 0.15
discount_amount = original_price * discount_rate
final_price = original_price - discount_amount
print(f"Item 1: Original price = ${original_price:.2f}, "
      f"Discount = ${discount_amount:.2f}, Final price = ${final_price:.2f}")

# Item 2
original_price = 59.90
discount_rate = 0.10
discount_amount = original_price * discount_rate
final_price = original_price - discount_amount
print(f"Item 2: Original price = ${original_price:.2f}, "
      f"Discount = ${discount_amount:.2f}, Final price = ${final_price:.2f}")
```

How to Read a Format String

```
new = 1 / 3 + 5  
print(f"{new:.2f}") # 5.33
```

- The `f` before the opening quote marks a formatted string (an f-string).

How to Read a Format String

```
new = 1 / 3 + 5  
print(f"{new:.2f}") # 5.33
```

- The `f` before the opening quote marks a formatted string (an f-string).
- Braces insert the value of `new`; the colon starts its format specification.

How to Read a Format String

```
new = 1 / 3 + 5  
print(f"{new:.2f}") # 5.33
```

- The `f` before the opening quote marks a formatted string (an f-string).
- Braces insert the value of `new`; the colon starts its format specification.
- In `.2f`, the final `f` selects fixed-point format, and `.2` sets the precision to 2 digits after the decimal point.

How to Read a Format String

```
new = 1 / 3 + 5  
print(f"{new:.2f}") # 5.33
```

- The `f` before the opening quote marks a formatted string (an f-string).
- Braces insert the value of `new`; the colon starts its format specification.
- In `.2f`, the final `f` selects fixed-point format, and `.2` sets the precision to 2 digits after the decimal point.
- With a precision but no type, as in `.2`, a `float` uses significant digits instead.

Why Format Strings Help

```
new = 1 / 3 + 5
print(new)          # 5.333333333333333
print(f"{new:.2}")  # 5.3: 2 significant digits
print(f"{new:.2f}") # 5.33: 2 decimal places

new = 5
print(f"{new:.2f}") # 5.00
print(f"{new}")     # 5: no format specification
```

Use `.2f` for consistent currency output, including trailing zeros.

Q2: Review the Q1 Code

- Do you notice anything repetitive?
- What parts are the same each time?
- What parts are different each time?
- If the logic had a bug, how many places would need to be fixed?
- What if we had 100 items instead of 4?

Q3: Rewrite Q1 Using a Function

Turn the repeated discount logic into a function and then call that function for each item.

Q3: A Function Version

```
def apply_discount(original_price, discount_rate):
```

```
    apply_discount(120.00, 0.15)
```

```
    apply_discount(59.90, 0.10)
```

```
    apply_discount(249.50, 0.20)
```

```
    apply_discount(19.99, 0.05)
```

Q3: A Function Version

```
def apply_discount(original_price, discount_rate):  
    discount_amount = original_price * discount_rate  
    final_price = original_price - discount_amount
```

```
apply_discount(120.00, 0.15)  
apply_discount(59.90, 0.10)  
apply_discount(249.50, 0.20)  
apply_discount(19.99, 0.05)
```

Q3: A Function Version

```
def apply_discount(original_price, discount_rate):  
    discount_amount = original_price * discount_rate  
    final_price = original_price - discount_amount  
    print(f"Original price = ${original_price:.2f}, "  
        f"Discount = ${discount_amount:.2f}, "  
        f"Final price = ${final_price:.2f}")
```

```
apply_discount(120.00, 0.15)  
apply_discount(59.90, 0.10)  
apply_discount(249.50, 0.20)  
apply_discount(19.99, 0.05)
```

Q4: Review the Function Version

- Which version is shorter and easier to read?
- Which version is easier to update if the logic changes?
- Which version is more organized and reusable?
- Which version would be easier to test and debug?

Q5: Improve the Program

Starting from the function version:

- show the discounted price for each item,
- compute the total price to pay,
- compute the total discount received.

Q5: The apply_discount Helper

```
def apply_discount(item_label, original_price, discount_rate):  
    discount_amount = original_price * discount_rate  
    final_price = original_price - discount_amount  
    print(f"{item_label}: Original price = ${original_price:.2f}, "  
          f"Discount = ${discount_amount:.2f}, Final price = ${final_price:.2f}")  
    return discount_amount, final_price
```

Q5: What Does a Function Return?

1. If a function has no `return` statement, what does it return?

Answer: `None`

Q5: What Does a Function Return?

1. If a function has no `return` statement, what does it return?
2. If a function returns just `final_price`, what kind of value is that?

Answer: one value, here a `float`

Q5: What Does a Function Return?

1. If a function has no `return` statement, what does it return?
2. If a function returns just `final_price`, what kind of value is that?
3. If a function returns `discount_amount`, `final_price`, what kind of value is that?

Answer: a tuple of two floats

Q5: Building `accumulate_total_and_discount`

```
def apply_discount(item_label, original_price, discount_rate):  
    discount_amount = original_price * discount_rate  
    final_price = original_price - discount_amount  
    print(f"{item_label}: Original price = ${original_price:.2f}, "  
        f"Discount = ${discount_amount:.2f}, Final price = ${final_price:.2f}")  
    return discount_amount, final_price  
  
def accumulate_total_and_discount(item_label, price, rate, total_discount, total_price):
```

Q5: Building `accumulate_total_and_discount`

```
def apply_discount(item_label, original_price, discount_rate):  
    discount_amount = original_price * discount_rate  
    final_price = original_price - discount_amount  
    print(f"{item_label}: Original price = ${original_price:.2f}, "  
        f"Discount = ${discount_amount:.2f}, Final price = ${final_price:.2f}")  
    return discount_amount, final_price  
  
def accumulate_total_and_discount(item_label, price, rate, total_discount, total_price):  
    discount_amount, final_price = apply_discount(item_label, price, rate)
```

Q5: Building accumulate_total_and_discount

```
def apply_discount(item_label, original_price, discount_rate):  
    discount_amount = original_price * discount_rate  
    final_price = original_price - discount_amount  
    print(f"{item_label}: Original price = ${original_price:.2f}, "  
        f"Discount = ${discount_amount:.2f}, Final price = ${final_price:.2f}")  
    return discount_amount, final_price  
  
def accumulate_total_and_discount(item_label, price, rate, total_discount, total_price):  
    discount_amount, final_price = apply_discount(item_label, price, rate)  
    total_discount += discount_amount  
    total_price += final_price  
    return total_discount, total_price
```

Q5: Better Function Names

- Good function names should say what the function does.
- A verb often helps: this function is updating running totals.
- `process_item` is vague; `accumulate_total_and_discount` is more specific.
- If a function only computes values, a name like `compute_item_summary` is clearer than `process_item`.

Q5: Driver Code

```
total_discount = 0
total_price = 0

total_discount, total_price = accumulate_total_and_discount("Item 1", 120.00, 0.15, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 2", 59.90, 0.10, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 3", 249.50, 0.20, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 4", 19.99, 0.05, total_discount, total_price)

print(f"Total price to pay: ${total_price:.2f}")
print(f"Total discount received: ${total_discount:.2f}")
```

Walk through the driver code one line at a time.

Q5: Driver Code

```
total_discount = 0
total_price = 0

total_discount, total_price = accumulate_total_and_discount("Item 1", 120.00, 0.15, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 2", 59.90, 0.10, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 3", 249.50, 0.20, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 4", 19.99, 0.05, total_discount, total_price)

print(f"Total price to pay: ${total_price:.2f}")
print(f"Total discount received: ${total_discount:.2f}")
```

Start `total_discount` at 0 while `total_price` is still 0.

Q5: Driver Code

```
total_discount = 0
total_price = 0

total_discount, total_price = accumulate_total_and_discount("Item 1", 120.00, 0.15, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 2", 59.90, 0.10, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 3", 249.50, 0.20, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 4", 19.99, 0.05, total_discount, total_price)

print(f"Total price to pay: ${total_price:.2f}")
print(f"Total discount received: ${total_discount:.2f}")
```

Now both running totals are initialized: `total_discount = 0` and `total_price = 0`.

Q5: Driver Code

```
total_discount = 0
total_price = 0

total_discount, total_price = accumulate_total_and_discount("Item 1", 120.00, 0.15, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 2", 59.90, 0.10, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 3", 249.50, 0.20, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 4", 19.99, 0.05, total_discount, total_price)

print(f"Total price to pay: ${total_price:.2f}")
print(f"Total discount received: ${total_discount:.2f}")
```

After Item 1: total_discount = 18.00, total_price = 102.00.

Q5: Driver Code

```
total_discount = 0
total_price = 0

total_discount, total_price = accumulate_total_and_discount("Item 1", 120.00, 0.15, total_discount,
total_discount, total_price = accumulate_total_and_discount("Item 2", 59.90, 0.10, total_discount,
total_discount, total_price = accumulate_total_and_discount("Item 3", 249.50, 0.20, total_discount,
total_discount, total_price = accumulate_total_and_discount("Item 4", 19.99, 0.05, total_discount,

print(f"Total price to pay: ${total_price:.2f}")
print(f"Total discount received: ${total_discount:.2f}")
```

After Item 2: total_discount = 23.99, total_price = 155.91.

Q5: Driver Code

```
total_discount = 0
total_price = 0

total_discount, total_price = accumulate_total_and_discount("Item 1", 120.00, 0.15, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 2", 59.90, 0.10, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 3", 249.50, 0.20, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 4", 19.99, 0.05, total_discount, total_price)

print(f"Total price to pay: ${total_price:.2f}")
print(f"Total discount received: ${total_discount:.2f}")
```

After Item 3: total_discount = 73.89, total_price = 355.51.

Q5: Driver Code

```
total_discount = 0
total_price = 0

total_discount, total_price = accumulate_total_and_discount("Item 1", 120.00, 0.15, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 2", 59.90, 0.10, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 3", 249.50, 0.20, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 4", 19.99, 0.05, total_discount, total_price)

print(f"Total price to pay: ${total_price:.2f}")
print(f"Total discount received: ${total_discount:.2f}")
```

After Item 4: total_discount = 74.89, total_price = 374.50.

Q5: Driver Code

```
total_discount = 0
total_price = 0

total_discount, total_price = accumulate_total_and_discount("Item 1", 120.00, 0.15, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 2", 59.90, 0.10, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 3", 249.50, 0.20, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 4", 19.99, 0.05, total_discount, total_price)

print(f"Total price to pay: ${total_price:.2f}")
print(f"Total discount received: ${total_discount:.2f}")
```

Now print the total price to pay: \$374.50. The total discount is still \$74.89.

Q5: Driver Code

```
total_discount = 0
total_price = 0

total_discount, total_price = accumulate_total_and_discount("Item 1", 120.00, 0.15, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 2", 59.90, 0.10, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 3", 249.50, 0.20, total_discount, total_price)
total_discount, total_price = accumulate_total_and_discount("Item 4", 19.99, 0.05, total_discount, total_price)

print(f"Total price to pay: ${total_price:.2f}")
print(f"Total discount received: ${total_discount:.2f}")
```

Now print the total discount received: \$74.89. The total price remains \$374.50.

Q5+: Add SGD and USD

Assume the prices are originally in SGD and use

$$1 \text{ SGD} = 0.74 \text{ USD}$$

```
USD_PER_SGD = 0.74
print(f"{'Item':<8}{'SGD':>10}{'USD':>10}")
print("--" * 28)
```

Q5+: Add SGD and USD

Assume the prices are originally in SGD and use

$$1 \text{ SGD} = 0.74 \text{ USD}$$

```
USD_PER_SGD = 0.74
print(f"{'Item':<8}{'SGD':>10}{'USD':>10}")
print("-" * 28)
for item_label, final_price_sgd in final_prices:
    final_price_usd = final_price_sgd * USD_PER_SGD
    print(f"{'item_label':<8}{'final_price_sgd':>10.2f}{'final_price_usd':>10.2f}")
```

Q5+: Get Items from a List

Adding an item should only require another entry in the data. Revise the code to get the items from a list using list comprehensions.

```
items = [("Item 1", 120.00, 0.15), ("Item 2", 59.90, 0.10),  
         ("Item 3", 249.50, 0.20), ("Item 4", 19.99, 0.05)]
```

- Write `compute_item_summary` to return the label, original price, discount amount, and final price.
- Use a list comprehension to create one summary per item.
- Print the summaries as a table with SGD and USD prices.

Q5+: List Comprehension Solution

```
USD_PER_SGD = 0.74
```

```
def compute_item_summary(item_label, price, rate):
```

```
    discount_amount = price * rate
```

```
    final_price = price - discount_amount
```

```
    return item_label, price, discount_amount, final_price
```

```
items = [("Item 1", 120.00, 0.15), ("Item 2", 59.90, 0.10),  
         ("Item 3", 249.50, 0.20), ("Item 4", 19.99, 0.05)]
```

```
rows = [compute_item_summary(item_label, price, rate)
```

```
        for item_label, price, rate in items]
```

```
print(f"{'Item':<8}{'Original':>10}{'Discount':>10}{'Final':>10}{'USD':>10}")
```

```
for item_label, original_sgd, discount_sgd, final_sgd in rows:
```

```
    print(f"{'item_label':<8}{'original_sgd':>10.2f}{'discount_sgd':>10.2f}"
```

```
          f"{'final_sgd':>10.2f}{'final_sgd * USD_PER_SGD':>10.2f}")
```

Q6: Use a While Loop Instead

Enhance the program so that it no longer uses hardcoded values.

- repeatedly ask for the item label
- ask for the original price
- ask for the discount rate
- stop when the user enters a special keyword such as `End`

Optional: error checking and case insensitivity.

Q6: One Suggested Structure

```
def apply_discount(item_label, original_price, discount_rate):  
    discount_amount = original_price * discount_rate  
    final_price = original_price - discount_amount  
    print(f"{item_label}: Original price = ${original_price:.2f}, "  
        f"Discount = ${discount_amount:.2f}, Final price = ${final_price:.2f}")  
    return discount_amount, final_price  
  
def accumulate_total_and_discount(item_label, price, rate, total_discount, total_price):  
    discount_amount, final_price = apply_discount(item_label, price, rate)  
    total_discount += discount_amount  
    total_price += final_price  
    return total_discount, total_price
```

Q6: Input Loop

```
total_discount = 0
total_price = 0

while True:
    item_label = input("Enter item name (or 'End' to finish): ")
    if item_label.lower() == "end":
        break
```

Q6: Input Loop

```
total_discount = 0
total_price = 0

while True:
    item_label = input("Enter item name (or 'End' to finish): ")
    if item_label.lower() == "end":
        break

    original_price = float(input("Enter original price: "))
    discount_rate = float(input("Enter discount rate: "))
```

Q6: Input Loop

```
total_discount = 0
total_price = 0

while True:
    item_label = input("Enter item name (or 'End' to finish): ")
    if item_label.lower() == "end":
        break

    original_price = float(input("Enter original price: "))
    discount_rate = float(input("Enter discount rate: "))

    total_discount, total_price = accumulate_total_and_discount(
        item_label, original_price, discount_rate,
        total_discount, total_price
    )
```

Q6: Input Loop

```
total_discount = 0
```

```
total_price = 0
```

```
while True:
```

```
    item_label = input("Enter item name (or 'End' to finish): ")
```

```
    if item_label.lower() == "end":
```

```
        break
```

```
    original_price = float(input("Enter original price: "))
```

```
    discount_rate = float(input("Enter discount rate: "))
```

```
    total_discount, total_price = accumulate_total_and_discount(  
        item_label, original_price, discount_rate,  
        total_discount, total_price  
    )
```

```
print(f"\nTotal discount received: ${total_discount:.2f}")
```

```
print(f"Total price to pay: ${total_price:.2f}")
```

Word Counts

- You have text in a variable called `document`
- Create a dictionary called `word_count` that has the words as keys and the number of times the word appears in `document` as the value

Word Counts

- You have text in a variable called `document`
- Create a dictionary called `word_count` that has the words as keys and the number of times the word appears in `document` as the value
- Use whitespace tokenization, so punctuation will still be attached to words

Word Counts

- You have text in a variable called `document`
- Create a dictionary called `word_count` that has the words as keys and the number of times the word appears in `document` as the value
- Use whitespace tokenization, so punctuation will still be attached to words
- Ignore case

Word Counts: Loop Solution

```
tokens = document.lower().split()
word_count = {}

for word in tokens:
    word_count[word] = word_count.get(word, 0) + 1
```

Using 0 as the default makes the first occurrence count as 1.

Word Counts: Dictionary Comprehension

```
tokens = document.lower().split()  
word_count = {word: tokens.count(word) for word in tokens}
```

Word Counts: Remove Punctuation

```
import string

tokens = [word.strip(string.punctuation).lower()
           for word in document.split()]
word_count = {}

for word in tokens:
    if word:
        word_count[word] = word_count.get(word, 0) + 1
```

Word Counts: Make a Function

Refactor the word-count code into `term_frequency(document)`.

- Take one document as the argument.
- Lowercase words, strip punctuation at their edges, and skip empty words.
- Return the dictionary of word counts.
- How can its keys tell us which words occur in this document?

Word Counts: Function Solution

```
import string

def term_frequency(document):
    word_count = {}
    for token in document.split():
        word = token.strip(string.punctuation).lower()
        if word:
            word_count[word] = word_count.get(word, 0) + 1
    return word_count

print(term_frequency("Cat cat dog!")) # {'cat': 2, 'dog': 1}
```

Dictionary keys are unique: cat is a key only once, even when its count is 2.

Document frequency

- TF-IDF measures how important a word is in a document relative to a corpus
- The “idf” is inverse document frequency: words appearing in many documents receive less weight
- Given a list of documents `corpus = [doc1 ...]`, compute the `document_frequency`
- The key should be the word, the value should be the number of documents it appears in
- Reuse `term_frequency(document)`: its keys contain each word once

Document frequency: Using term_frequency

```
document_frequency = {}  
  
for doc in corpus:  
    for word in term_frequency(doc):  
        document_frequency[word] = document_frequency.get(word, 0) + 1
```

Iterating over a dictionary visits its unique keys, so each word adds 1 per document, regardless of its word count.

Inverse Document Frequency

Use the document counts to compute IDF for each word:

$$\text{idf}(w) = \log \left(\frac{N}{\text{df}(w)} \right), \quad N = \text{number of documents.}$$

```
import math

idf = {word: math.log(len(corpus) / count)
       for word, count in document_frequency.items()}
```

A word in every document has IDF 0; a word in fewer documents has a larger IDF.

Takeaways

- Functions help us avoid repeating the same logic many times.
- Returning values lets one function support a larger workflow.
- Small helper functions make later extensions easier.
- Formatting strings carefully makes numerical output easier to read.